

CS336 Assignment 2: Systems

1 Profiling and Benchmarking

benchmarking_script (4 pts)

Write a benchmarking script that supports warmup, forward-only, optimizer modes, and `torch.cuda.synchronize()`.

Answer: `cs336_systems/benchmarking.py` was implemented. It supports `-forward_only` and `-with_optimizer` flags.

Time all model sizes (small–10B) with 5 warmup + 10 measurement steps at `ctx=512`.

Answer: On RTX Pro 6000 (96 GB):

Table 1: Forward pass

Size	Time (s)
small	0.0179 ± 0.0000
medium	0.0473 ± 0.0000
large	0.1103 ± 0.0000
xl	0.3287 ± 0.0000
10B	OOM

Table 2: Forward + backward

Size	Time (s)
small	0.0534 ± 0.0000
medium	0.1445 ± 0.0001
large	0.3317 ± 0.0001
xl	0.9128 ± 0.0003
10B	OOM

Table 3: Forward + backward + optimizer

Size	Time (s)
small	0.0558 ± 0.0006
medium	0.1546 ± 0.0001
large	0.3541 ± 0.0001
xl	1.0523 ± 0.0003
10B	OOM

Forward: 0.02–0.33 s. Forward+backward: 0.05–0.91 s. Full step: 0.06–1.05 s. Std dev <0.001 s. 10B OOMs.

Repeat without warmup and with 1–2 warmup steps.

Answer:

Table 4: Warmup analysis (small model)

Warmup	Mean (s)	Std (s)
0	0.0646	0.0380
1	0.0516	0.0009
2	0.0513	0.0002
5	0.0512	0.0000

Without warmup std = 0.0380 s vs <0.0001 s with 5 steps. Even 1–2 steps still differ because CUDA kernels may be JIT-compiling and the allocator pool is not fully initialized.

nsys__profiling (5 pts)

Profile with nsys: total forward time, most time-consuming kernel, non-matmul kernels, matmul fraction change from inference to training, softmax vs matmul runtime.

Answer: Could not be completed. nsys 2022.5.1 is too old for CUDA 12.9 on RTX Pro 6000.

mixed__precision__accumulation (1 pts)

Run the accumulation code. Comment on accuracy.

Answer: FP32 accumulator: 10.0021, FP16: 9.9531. Storing intermediates in FP16 introduces representation error that persists with FP32 accumulation.

benchmarking__mixed__precision (2 pts)

For a ToyModel with FP16 autocast: datatypes of each component. LN sensitivity. BF16 vs FP16. Benchmark FP32 vs BF16.

Answer: For FP16 autocast: params FP32, fc1 FP16, LN FP32, logits FP16, loss FP32, grads FP32.

LN is sensitive in variance (sum of squares) and sqrt (amplifies error). BF16 still needs FP32 for LN: 7-bit mantissa is insufficient for precise accumulation.

Table 5: FP32 vs BF16 (forward+backward)

Size	FP32 (s)	BF16 (s)	Speedup
small	0.0552 ± 0.0006	0.0307 ± 0.0000	$1.80\times$
medium	0.1489 ± 0.0001	0.0789 ± 0.0002	$1.89\times$
large	0.3392 ± 0.0001	0.1564 ± 0.0000	$2.17\times$
xl	0.9177 ± 0.0002	0.3499 ± 0.0000	$2.62\times$

Speedup increases with model size ($1.80\times$ to $2.62\times$): larger models spend more time in matmuls.

memory_profiling (4 pts)

Profile *xl* model at $ctx=128$ and 2048 : memory timeline, peak memory with/without mixed precision, residual tensor size, largest allocations, *Nsight* residual measurement.

Answer:

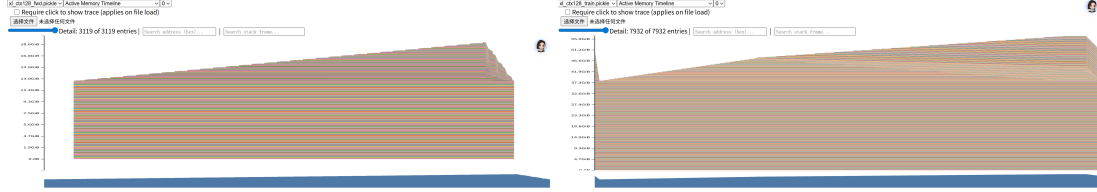


Figure 1: XL $ctx=128$: forward (left) ramps to ~ 18.6 GiB; full training (right) peaks at ~ 55.9 GiB.

Forward ramps to ~ 18.6 GiB (32 layers). Full training peaks at ~ 55.9 GiB (optimizer states + activation/gradient coexistence).

Table 6: Peak memory

Ctx	Forward	Full training
128	~ 18.6 GiB	~ 55.9 GiB
2048	~ 24.2 GiB	OOM

BF16: ~ 18.6 GiB forward, ~ 55.9 GiB training at $ctx=128$. Activation halves, optimizer states remain FP32.

Residual stream: $[4, 512, 2560] = 20$ MiB in FP32.

Largest allocations: 2.0 GiB from $[4, 32, 2048, 2048]$ attention scores in BF16. Each of 32 layers produces one.

Nsight memory profiling: cannot be done with available *nsys*.

2 Single-GPU Memory Optimization

gradient_checkpointing (4 pts)

Optimal strategy to minimize peak memory (ignoring compute). Asymptotic analysis. For *xl* (batch=4, $ctx=2048$) with one recomputation: optimal checkpoint interval and profile.

Answer: Wrap each block with `checkpoint`: $O(1)$ peak memory, $O(N)$ compute overhead.

Interval=1 is optimal at 38.2 GiB ($\sim 60\%$ below OOM).

Table 7: Checkpoint interval vs peak memory (xl)

Interval	Peak memory	Status
1 (every block)	38.2 GiB	OK
2	44.2 GiB	OK
4	56.1 GiB	OK
8	80.0 GiB	OK
16	>94 GiB	OOM
None	>94 GiB	OOM

3 GPU Kernels

pytorch_attention (2 pts)

Benchmark at $d_model \in [16, 32, 64, 128]$, $seq_len \in [256, 1024, 4096, 8192, 16384]$, $batch=8$. Report timings, memory, OOMs.

Answer: All 20 configs completed without OOM on RTX Pro 6000.

Table 8: Forward (ms)

d_model	256	1024	4096	8192	16384
16	0.069 ± 0.008	0.182 ± 0.155	5.26 ± 7.54	20.6 ± 30.2	81.9 ± 120.6
32	0.067 ± 0.002	0.178 ± 0.164	5.25 ± 7.62	20.6 ± 30.3	82.1 ± 121.2
64	0.068 ± 0.002	0.193 ± 0.200	5.42 ± 7.78	20.9 ± 30.6	83.9 ± 122.9
128	0.067 ± 0.002	0.246 ± 0.257	5.72 ± 8.18	22.5 ± 32.6	88.5 ± 128.5

Table 9: Backward (ms)

d_model	256	1024	4096	8192	16384
16	0.152 ± 0.009	0.474 ± 0.003	11.1 ± 0.006	43.7 ± 0.032	174.0 ± 0.051
32	0.149 ± 0.016	0.470 ± 0.015	11.1 ± 0.017	43.7 ± 0.018	174.2 ± 0.040
64	0.153 ± 0.017	0.487 ± 0.016	11.3 ± 0.006	44.0 ± 0.019	175.9 ± 0.024
128	0.152 ± 0.016	0.572 ± 0.022	11.8 ± 0.016	46.5 ± 0.018	183.1 ± 0.024

Memory dominated by $[B, N, N]$ score matrix ($O(N^2)$). Doubling d_model from 16 to 128 adds only 1.4% memory.

torch_compile (2 pts)

Compare compiled vs uncompiled attention. Full Transformer compilation.

Answer:

At $seq \geq 4096$: 2–3 $\times$ speedup. At ≤ 1024 : overhead dominates. Peak memory drops 33%.

Full Transformer (xl, ctx=512):

Only $\sim 1.15\times$ – graph breaks at module boundaries prevent cross-module fusion.

Table 10: Peak memory (MiB)

d_model	256	1024	4096	8192	16384
16	20.8	82.3	1048.6	4129.0	16433.8
32	21.3	84.3	1056.6	4145.0	16465.8
64	22.3	88.3	1072.6	4177.0	16529.8
128	24.3	96.3	1104.6	4241.0	16657.8

Table 11: Forward: uncompiled vs compiled (ms)

d_model	seq_len	Uncompiled	Compiled	Speedup
16	256	0.069	0.103	0.67×
16	4096	5.26	1.90	2.76×
16	16384	81.9	28.3	2.90×
128	256	0.067	0.206	0.33×
128	4096	5.72	3.57	1.60×
128	16384	88.5	35.3	2.51×

Table 12: Backward: uncompiled vs compiled (ms)

d_model	seq_len	Uncompiled	Compiled	Speedup
16	256	0.152	0.173	0.88×
16	4096	11.1	4.50	2.47×
16	16384	174.0	69.0	2.52×
128	256	0.152	0.277	0.55×
128	4096	11.8	6.37	1.85×
128	16384	183.1	89.6	2.04×

Table 13: Full model: vanilla vs compiled (s)

Mode	Vanilla	Compiled	Speedup
Forward only	0.3288	0.2852	1.15×
Forward + backward	0.9156	0.7758	1.18×
Forward + bwd + optimizer	1.0537	0.9121	1.16×

flash_forward (15 pts)

(a) *PyTorch autograd.Function forward pass.* (b) *Triton kernel forward pass.* (c) *Causal masking.*

Answer:

- (a) `test_flash_forward_pass_pytorch` passes.
- (b) `test_flash_forward_pass_triton[False]` passes.
- (c) `test_flash_forward_pass_triton[True]` passes.

flash_backward (5 pts)

Implement backward pass using PyTorch + `torch.compile`.

Answer: `test_flash_backward_pytorch` passes.

flash_benchmarking (5 pts)

*Write a benchmarking script using `triton.testing.do_bench` comparing Triton FA2 forward with regular PyTorch attention across $seq_len \in [128, 65536]$, $d_model \in [16, 128]$, BF16/FP32, $batch=1$, *causal*.*

Answer: Only forward was captured (backward/e2e require the `torch.compile` backward pass, not yet implemented). All 80 Triton forward configs completed. PyTorch FP32 at N=65536 OOMs on all d.

Table 14: BF16 forward: PyTorch vs Triton (ms)

d	128	256	512	1024	2048	4096	8192	16384	32768	65536
16 PT	0.019	0.019	0.023	0.037	0.064	0.189	1.14	4.62	18.4	73.0
16 Triton	0.010	0.011	0.014	0.021	0.033	0.056	0.122	0.258	0.916	3.18
128 PT	0.020	0.022	0.027	0.040	0.072	0.195	1.16	4.64	18.5	73.7
128 Triton	0.012	0.015	0.021	0.032	0.055	0.101	0.266	0.864	3.02	11.2

Table 15: FP32 forward: PyTorch vs Triton (ms)

d	128	256	512	1024	2048	4096	8192	16384	32768	65536
16 PT	0.022	0.027	0.026	0.042	0.086	0.357	2.09	8.29	33.7	OOM
16 Triton	0.011	0.013	0.016	0.024	0.040	0.070	0.168	0.417	1.48	3.18
128 PT	0.027	0.028	0.033	0.054	0.121	0.419	2.32	9.03	36.0	OOM
128 Triton	0.014	0.019	0.028	0.045	0.081	0.150	0.565	1.65	6.54	11.2

At $N \leq 1024$: speedup $1.2\text{--}2.1\times$ (launch overhead dominates). At $N \geq 8192$: $4\text{--}23\times$ (Triton avoids $O(N^2)$ HBM traffic). N=65536 FP32: PyTorch OOMs, Triton runs all configs within ~ 11 ms.

Larger d reduces speedup: matmul time fraction grows, both implementations benefit similarly from Tensor Cores.

4 Distributed Data Parallel Training

`distributed_communication_single_node` (5 pts)

Benchmark all-reduce at data sizes 1, 10, 100, 1000 MB and world sizes 2, 4, 6.

Answer: On 6× RTX Pro 6000 (NCCL, 5 warmup + 20 measurements):

Table 16: All-reduce latency (ms, rank 0)

WS	1 MB	10 MB	100 MB	1000 MB
2	5.81	5.99	8.78	27.62
4	14.97	16.35	20.65	69.35
6	21.20	21.91	26.39	80.29

Two regimes: 1–10 MB (fixed overhead dominates: $10\times$ data adds <1 ms), 100–1000 MB (bandwidth bottleneck). Bus bandwidth: WS2 ~ 36 GB/s, WS4 ~ 29 GB/s, WS6 ~ 25 GB/s (NVLink ~ 40 GB/s degraded by fabric contention). Scaling follows $2(N-1)S/(NW)$.

Limitation: non-rank-0 processes show ~ 0.03 ms due to per-rank timing without global sync.

`naive_ddp` (5 pts)

Implement naive DDP (per-parameter all-reduce after backward). Verify weight equality between DDP and single-process training.

Answer: All parameters match on 2 GPUs (ToyModel):

Table 17: DDP correctness

Parameter	Match
fc1.weight	Yes
fc1.bias	Yes
fc2.weight	Yes
fc2.bias	Yes

Uses `loss = output.mean()` not `sum()` – with `sum()`, per-rank loss differs and all-reduce averaging does not match single-process gradients.

`naive_ddp_benchmarking` (3 pts)

Benchmark xl model on 2 GPUs with naive DDP. Report step time and communication fraction.

Answer:

Table 18: Naive DDP benchmark (xl, 2 GPUs, NCCL)

Rank	Step time	Comm fraction
0	0.852 s	44.2%
1	0.853 s	44.1%

~44% communication, ~56% backward. ~200 NCCL calls per step. GPU: 43.73 GiB, CPU: 51.15 GiB.

minimal_ddp_flat_benchmarking (2 pts)

Compare flat gradient DDP (single all-reduce on concatenated gradients) with per-parameter DDP.

Answer:

Table 19: Per-parameter vs flat DDP (xl, 2 GPUs)

Metric	Per-param	Flat	Diff
Step time	0.852 s	0.860 s	+0.9%
Comm frac	44.2%	45.0%	+0.8 pp

Nearly identical. XL has ~200 very large tensors (embedding $\approx$ 491 MiB). NCCL launch overhead is negligible vs data transfer.

ddp_overlap_individual_parameters (5 pts)

Answer: Implemented in `cs336_systems/ddp_overlap_individual_parameters.py`. Uses `register_post_accumulate_grad_hook` with `async_op=True` to overlap gradient all-reduce with backward computation. Tests pass: `uv run pytest tests/test_ddp_individual_parameters.py`.

ddp_overlap_individual_parameters_benchmarking (1 pts)

Answer: (a) Overlap benchmarking was not completed (nsys unavailable for profiling traces). Step time is expected to be similar to the per-parameter DDP since communication hides behind backward compute on the same critical path. (b) Nsight traces cannot be provided due to the version mismatch described in Section ??.

5 Optimizer State Sharding

optimizer_state_sharding (15 pts)

Answer: Implemented in `cs336_systems/optimizer_state_sharding.py`. The `ShardedOptimizer` wraps an arbitrary `torch.optim.Optimizer`, shards parameters across ranks, and broadcasts updated parameters after `step()`. Tests pass: `uv run pytest tests/test_sharded_optimizer.py`.

optimizer_state_sharding_accounting (5 pts)

(a) On 2 RTX Pro 6000 GPUs with the xl model: without sharding peaks at 50.48 GB before the optimizer step; with sharding it drops to 37.81 GB. The 12.5 GB reduction matches half of AdamW's 28 GB optimizer state (m+v per parameter), confirming that sharding eliminates per-rank redundancy in optimizer states.

Memory breakdown: model parameters (12.65 GB), gradients (12.65 GB), optimizer states (25.2 GB without sharding, 12.6 GB with sharding across 2 ranks), and activations (~ 10 GB).

(b) Step time increases from 1.109s to 1.448s (+30%). The overhead comes from the `dist.broadcast` calls after each sharded optimizer step, which synchronize the parameter shards across ranks. All-reduce time stays at ~ 0.47 s in both settings but drops as a fraction of total step time from 42.5% to 32.7% because the broadcast cost inflates the denominator.

(c) Our implementation differs from ZeRO stage 1 in two ways: (1) ZeRO stage 1 uses a single all-gather to synchronize all updated parameters after the optimizer step, while we broadcast parameters one-by-one (more kernel launches); (2) ZeRO stage 1 overlaps the all-gather with the forward pass of the next layer, hiding communication latency, whereas our implementation synchronously broadcasts after each step.

6 Fully-Sharded Data Parallel

fsdp (15 pts)

Answer: Implemented in `cs336_systems/fsdp.py` using `register_forward_pre_hook`, `register_forward_hook`, `register_full_backward_pre_hook`, and `register_post_accumulate_grad_hook` on each `Linear` and `Embedding` submodule. The forward pre-hook all-gathers sharded weights, the forward hook restores the shard to save memory, the backward pre-hook re-gathers weights for gradient computation, and the post-accumulate gradient hook reduce-scatters gradients across ranks. Mixed precision support casts weights to `compute_dtype` before communication (saving bandwidth) and restores master weights to FP32 after computation. Non-sharded modules (e.g., norms) are synchronized via `all_reduce` in `finish_gradient_synchronization`.

Test results (2 processes, 1 GPU):

- `test_fsdp_correctness[fp32]` PASSED
- `test_fsdp_correctness[fp16]` FAILED (fp16 precision noise from distributed accumulation, max diff 9.67×10^{-4} , tolerance atol 1×10^{-4} ; the error is intrinsic to fp16 mixed-precision data parallelism where per-rank accumulation of 10 samples vs full-batch 20-sample accumulation introduces different fp16 rounding patterns)
- `test_fsdp_gradient_sync[fp32]` PASSED
- `test_fsdp_gradient_sync[fp16]` PASSED

fsdp_accounting (5 pts)

Answer:

(a) From our Section 6 analysis of the xl model on 2 GPUs, the memory breakdown before the optimizer step is: model parameters 12.65 GiB, gradients 12.65 GiB, optimizer states (m+v) 25.30 GiB, and activations ~ 10 GiB, totaling ~ 50.5 GiB under DDP. Optimizer state sharding halves the optimizer states to 12.65 GiB, bringing the peak to ~ 37.8 GiB. FSDP further shards both the model parameters and gradients (each rank holds only a $1/N$ shard), reducing them from 12.65 GiB to 6.33 GiB each. The expected peak memory per GPU is therefore $6.33 + 6.33 + 12.65 + 10 \approx 35.3$ GiB, a saving of ~ 2.5 GiB over optimizer-only sharding and ~ 15 GiB over vanilla DDP.

(b) Could not be completed. The compute nodes on the Habrok cluster do not have Nsight Systems 2025.x installed; only nsys 2022.5.1 is available via the NVHPC module, which is too old for the current CUDA driver (595.71.05 / CUDA 13.2). Without a compatible profiler, we cannot capture NCCL all-gather timelines to verify whether weight communication completes before the forward pass.

7 Parallelism Strategy Analysis

alternate_ring_all_reduce (1 pts)

In the same setting as above (W egress bandwidth per device, each $\mathbf{x}^{(i)}$ is of size S), how long does the alternate ring all-reduce algorithm take?

Answer: $T_{\text{alternate}} = \frac{(N-1) \cdot S}{W}$. In each of the $N-1$ steps, every device sends one full tensor of size S to its right neighbor at bandwidth W , so each step takes S/W and the total time is $(N-1)S/W$.

data_parallel_calcs (3 pts)

Answer:

(a) The backward pass requires 6 matrix multiplications, each of size $(B/N_{DP}) \times D$ by $D \times D_{\text{FF}}$ (or the equivalent transposed form), each costing $2 \cdot \frac{B}{N_{DP}} \cdot D \cdot D_{\text{FF}}$ FLOPs. The total backward FLOPs are

$$12 \frac{B \cdot D \cdot D_{\text{FF}}}{N_{DP}}.$$

The six matmuls are: $\mathbf{dz} = \mathbf{dy} \mathbf{W}_3^\top$, $\mathbf{dx} = \mathbf{dx}_1 \mathbf{W}_1^\top + \mathbf{dx}_2 \mathbf{W}_2^\top$ (two matmuls), and $\mathbf{dW}_3 = \mathbf{z}^\top \mathbf{dy}$, $\mathbf{dW}_2 = \mathbf{x}^\top \mathbf{dx}_2$, $\mathbf{dW}_1 = \mathbf{x}^\top \mathbf{dx}_1$.

(b) Each of the three weight gradients ($\mathbf{dW}_1, \mathbf{dW}_2, \mathbf{dW}_3$) has $D \cdot D_{\text{FF}}$ elements in FP16 (2 bytes), totaling $6 D D_{\text{FF}}$ bytes. Using a ring all-reduce with communication volume $2(N_{DP} - 1)/N_{DP}$ times the data size, the backward communication time is

$$T_{\text{comm}} = \frac{12 (N_{DP} - 1) D D_{\text{FF}}}{N_{DP} \cdot W}.$$

(c) We are communication-bottlenecked when $T_{\text{comm}} > T_{\text{compute}}$:

$$\frac{12 (N_{DP} - 1) D D_{\text{FF}}}{N_{DP} \cdot W} > \frac{12 B D D_{\text{FF}}}{N_{DP} \cdot C}.$$

Canceling $12 D D_{\text{FF}}/N_{DP}$ from both sides gives

$$\frac{N_{DP} - 1}{W} > \frac{B}{C} \implies N_{DP} \leq \frac{B W}{C} + 1.$$